

Aula 06

Camadas, validação e erros

Como se organiza código em empresa

Seu `server.js` funciona

E tem 250 linhas.

E faz, ao mesmo tempo:

- interpretar HTTP
- validar dados
- aplicar regras de negócio
- guardar e buscar dados

Quatro trabalhos. Uma bancada só.

O que isso custa na prática

Quero...	Hoje eu preciso...
trocar array por banco	mexer em todas as rotas
reusar uma regra num script	impossível, está colada no <code>res</code>
testar a validação	subir o servidor inteiro
trabalhar em dupla	conflito de merge garantido



A cozinha do restaurante

Ninguém monta uma cozinha onde a mesma pessoa recebe o cliente, cozinha, controla o estoque e lava a louça — na mesma bancada.

Garçom · Cozinheiro · Estoquista

Trocou de fornecedor?

Só o estoquista precisa saber.

As camadas

Requisição HTTP

↓

ROUTES

"que rotas existem?"

CONTROLLER

"traduz HTTP ↔ negócio"

SERVICE

"quais são as regras?"

← o cérebro

REPOSITORY

"onde os dados ficam?"

↓

Array / Banco

A regra que amarra tudo

Camada	Sabe HTTP?	Sabe onde ficam os dados?
Routes	✓	✗
Controller	✓	✗
Service	✗	✗
Repository	✗	✓

● **res.status()** dentro de um service = errado

Repository — só dados

```
export const tarefaRepository = {  
  listarTodas: () => tarefas,  
  buscarPorId: (id) =>  
    tarefas.find((t) => t.id === id),  
  criar: (dados) => { ... },  
  remover: (id) => { ... },  
};
```

Nenhuma regra. Só entrada e saída.

Service — as regras

```
criar(dados) {  
  const erros = validar(dados);  
  if (erros.length) throw new ErroDeValidacao(erros);  
  
  if (repo.existeComTitulo(dados.titulo))  
    throw new ErroDeConflito("já existe");  
  
  return repo.criar({ ...dados, concluida: false });  
}
```

Não sabe o que é `req`, `res` ou `status`.

Poderia ser chamado por um script de terminal.

Controller — o tradutor

```
criar(req, res, next) {  
  try {  
    const nova = tarefaService.criar(req.body);  
    res.status(201).json(nova);  
  } catch (erro) {  
    next(erro);    // manda para o tratador  
  }  
}
```

Três linhas. Sempre o mesmo formato.

Routes — só o mapa

```
router.get("/", controller.listar);  
router.post("/", controller.criar);  
router.get("/:id", controller.buscarPorId);  
router.patch("/:id", controller.atualizar);  
router.delete("/:id", controller.remover);
```

Em 5 linhas você entende a API inteira.

← **Esse é o ganho**

express.Router()

```
// routes/tarefa.routes.js
const router = Router();
router.get("/", ...); // sem repetir /tarefas

// app.js
app.use("/tarefas", tarefaRoutes);
//           ↑ o prefixo aparece UMA vez
```

Novo recurso = um arquivo + uma linha.

Erros customizados

```
class ErroDaAplicacao extends Error {  
    constructor(mensagem, status) {  
        super(mensagem);  
        this.status = status;  
    }  
}  
  
class ErroNaoEncontrado extends ErroDaAplicacao {  
    constructor(msg) { super(msg, 404); }  
}
```

O service diz **o que** aconteceu,
não **qual status** devolver.

Calma: o que é `class`?

Um **molde** para criar objetos parecidos.

```
class Pessoa {  
    constructor(nome) {    // roda ao criar  
        this.nome = nome;  // this = o objeto sendo criado  
    }  
}  
  
const ana = new Pessoa("Ana");    // new = "crie um"  
ana.nome                          // "Ana"
```

É a única vez que usamos `class` no módulo.



extends e super

```
class ErroNaoEncontrado extends ErroDaAplicacao {  
    //          ↑ "herda tudo de"  
    constructor(mensagem) {  
        super(mensagem, 404);  
        //  ↑ chama o constructor do PAI  
    }  
}
```

`extends` = aproveita o que o outro já tem

`super()` = "pai, faça a sua parte primeiro"

Por que usar **Error** como base?

```
class ErroDaAplicacao extends Error { ... }  
//  
//           ↑  
//           o Error nativo do JavaScript
```

Herdando de **Error**, nosso erro:

- funciona com **throw**
- é capturado por **catch**
- tem **.message** e **.stack** de graça



instanceof

```
erro instanceof ErroDeValidacao // true / false
```

Pergunta: "este objeto foi criado a partir daquele molde?"

É assim que o tratador decide o status:

```
if (erro instanceof ErroDeValidacao) return res.status(400)...  
if (erro instanceof ErroDaAplicacao) return res.status(erro.status)...  
res.status(500)... // não é nenhum dos nossos: bug
```

Middleware de erro: 4 parâmetros

```
export function tratadorDeErros(erro, req, res, next) {  
  console.error(erro);  
  
  if (erro instanceof ErroDeValidacao)  
    return res.status(400).json({ erros: erro.erros });  
  
  if (erro instanceof ErroDaAplicacao)  
    return res.status(erro.status).json({ erro: erro.message });  
  
  res.status(500).json({ erro: "Erro interno" });  
}
```

O `erro` na 1ª posição é o que o marca como tratador.

Ordem importa

```
app.use("/tarefas", tarefaRoutes);  
app.use(rotaNaoEncontrada);    // 404  
app.use(tratadorDeErros);      // SEMPRE o último
```

Nunca exponha o erro interno

```
res.status(500).json({ erro: "Erro interno do servidor" });
```

Não devolva `erro.stack` ao cliente.

É informação para **atacante**.

Registre no log. Responda genérico.

! Pegadinha do async

Em código assíncrono, um `throw` solto **não chega** no middleware de erro.

```
async criar(req, res, next) {  
  try {  
    ...await...  
  } catch (erro) {  
    next(erro); // obrigatório  
  }  
}
```

Você vai sentir isso na aula 07.

Variáveis de ambiente

```
# .env – NUNCA vai para o Git  
PORTA=3000  
NODE_ENV=desenvolvimento
```

```
"dev": "node --watch --env-file=.env src/server.js"
```

```
const PORTA = process.env.PORTA ?? 3000;
```

E sempre o `.env.example`

```
# .env.example – este VAI para o Git  
PORTA=  
NODE_ENV=
```

Quem clonar o projeto sabe o que configurar.

`.gitignore` com `.env` **no mesmo minuto**
em que o arquivo nasce. Nunca depois.

A estrutura final

```
src/  
├── server.js           sobe o servidor  
├── app.js             monta o Express  
├── routes/  
├── controllers/  
├── services/  
├── repositories/  
├── validacoes/  
├── middlewares/  
└── erros/
```

"Mas são muito mais arquivos!"

Sim. E cada um é **pequeno**,
tem um nome que **diz o que faz**,
e você abre só o que precisa.

"Onde está a regra de duplicidade?"

→ `services/`. Sempre.

A prova de que valeu a pena

Semana que vem trocamos o array
por um **banco SQLite**.

Só o **repository** vai mudar.

Controllers, services, rotas, validações:
intactos.

Recapitulando

1. Rota declara · controller traduz · service decide · repository guarda
2. Service nunca vê `req / res` — ele lança erros, o middleware traduz
3. Configuração no `.env`, que **nunca** vai para o Git

Mão na massa 🚀

Refatore sua API de filmes em camadas

⚠️ É **pré-requisito** para a aula 07